

AI Sandbox — Master & Phase Prompts

Handoff prompt pack for the Cloud-Based Ephemeral Sandbox Platform for AI Agents

MASTER PROMPT

You are the senior software engineer responsible for continuing the project "Cloud-Based Ephemeral Sandbox Platform for AI Agents."

Repository:

<https://github.com/tanmay3114/ai-sandbox>

Current verified baseline:

- Phase 1 complete
- Phase 2 complete
- Phase 3 complete
- Baseline commit: bae5f51
- Python/FastAPI/PostgreSQL/SQLAlchemy/Alembic/Docker
- Treat the existing repository as the source of truth.

Before changing anything:

1. Inspect the repository structure, README, pyproject.toml, migrations, application code, and tests.
2. Understand the existing architecture and security profile.
3. Do not rebuild completed phases.
4. Identify the smallest safe change required for the requested phase.
5. Preserve backward compatibility unless the phase explicitly requires a change.

Non-negotiable security invariants:

- Every execution uses a fresh ephemeral Docker container.
- network_mode="none".
- read_only=True.
- user="10001:10001".
- cap_drop=["ALL"].
- no-new-privileges=true.
- memory limit 256 MB.
- CPU approximately 0.5 core.
- PID limit 32.
- restricted 32 MB /tmp with noexec,nosuid,nodev.
- no host mounts.
- no Docker socket exposed to sandbox code.
- no arbitrary image selection.
- bounded stdout/stderr.
- execution timeout is enforced.
- execution containers are cleaned up on success, failure, and timeout.
- cleanup must not touch unrelated containers.
- secrets must never be committed.

Persistence/lifecycle invariants:

- Sandbox is a logical PostgreSQL resource, not a Docker container.
- One sandbox can have many execution jobs.
- Each job gets a fresh execution container.
- Sandbox TTL and execution timeout are separate concepts.
- Explicit lifecycle/state transitions must remain valid.
- Existing recovery/reconciliation behavior must not be silently removed.

Engineering requirements:

- Prefer existing abstractions over duplication.
- Do not introduce Redis, Celery, Kafka, Kubernetes, or other major infrastructure unless the phase explicitly requires it.
- Add or update tests for every behavioral change.
- Run pytest and Ruff.
- Review git diff for accidental changes and secrets.
- Explain important architectural tradeoffs.
- Do not claim completion until tests and verification pass.

At the end, report:

1. What changed.
2. Why it changed.
3. Files changed.
4. Tests run and results.

5. Security verification.
6. Any limitations or follow-up work.

PHASE 1 — SECURE EPHEMERAL EXECUTION

PHASE 1 – SECURE EPHEMERAL EXECUTION

Goal:

Build the foundational secure execution engine for untrusted Python code using Docker.

Requirements:

- One request/execution creates one fresh Docker container.
- Use a controlled Python 3.12-slim execution image.
- Execute supplied Python code.
- Enforce execution timeout.
- Enforce memory limit of 256 MB.
- Enforce CPU limit of approximately 0.5 core.
- Enforce PID limit of 32.
- Disable networking with `network_mode="none"`.
- Use a read-only root filesystem.
- Run as non-root user `10001:10001`.
- Drop all Linux capabilities.
- Enable no-new-privileges.
- Provide restricted 32 MB `/tmp` with `noexec,nosuid,nodev`.
- Do not mount the host filesystem.
- Do not expose the Docker socket.
- Bound stdout/stderr and report truncation.
- Guarantee container cleanup in all paths, including timeout and exceptions.
- Use labels so only platform-owned containers are targeted for cleanup.
- Add an application-level concurrency limit.
- Add unit, integration, and security tests.
- Do not introduce PostgreSQL, Redis, Celery, Kubernetes, or cloud deployment yet.

Definition of done:

- Normal code executes successfully.
- Infinite loops time out and do not leak containers.
- Security restrictions are tested.
- Unrelated Docker containers are unaffected.
- pytest passes.
- Ruff passes.

PHASE 2 — FASTAPI API LAYER

PHASE 2 – FASTAPI API LAYER

Goal:

Expose the Phase 1 execution engine through a clean HTTP API.

Requirements:

- Build the FastAPI application.
- GET /health reports process health.
- GET /ready verifies required infrastructure, especially Docker.
- POST /api/v1/sandboxes/execute accepts code and timeout configuration and returns a structured execution result.
- Use typed request/response schemas.
- Validate input.
- Map expected failures to controlled HTTP errors.
- Do not expose tracebacks, Docker internals, filesystem paths, credentials, or secrets.
- Run blocking Docker operations through an appropriate threadpool/offloading mechanism so API event-loop responsiveness is preserved.
- Preserve all Phase 1 security guarantees.
- Add API tests, timeout tests, error tests, and security regression tests.
- Do not introduce PostgreSQL, Redis, Celery, Kubernetes, authentication, or cloud deployment yet.

Definition of done:

- Health/readiness endpoints work.
- Successful code execution works through HTTP.
- Timeout behavior works through HTTP.
- Errors are sanitized.
- No sandbox container leaks.
- Existing Phase 1 tests still pass.
- pytest and Ruff pass.

PHASE 3 — POSTGRESQL, PERSISTENCE, AND SANDBOX LIFECYCLE

PHASE 3 — POSTGRESQL, PERSISTENCE, AND SANDBOX LIFECYCLE

Goal:

Introduce durable Sandbox and ExecutionJob metadata while preserving the hardened ephemeral execution model.

Requirements:

- Add PostgreSQL.
- Use SQLAlchemy 2.x.
- Use Alembic migrations.
- Add Sandbox and ExecutionJob models.
- A Sandbox is a persistent logical resource, not a Docker container.
- One Sandbox has many ExecutionJobs.
- Every ExecutionJob still executes in a fresh ephemeral Docker container.
- Add explicit execution state transitions.
- Distinguish Sandbox TTL from individual execution timeout.
- Support sandbox creation, inspection, and deletion.
- Expired sandboxes must reject new executions while remaining inspectable until destroyed.
- Persist execution outcomes including success, timeout, and infrastructure failure.
- Prevent concurrent execution claims from corrupting job state.
- Preserve targeted Docker cleanup using sandbox/job labels.
- Add startup reconciliation for expired sandboxes and interrupted execution/destroying states.
- Keep Docker security profile unchanged.
- Add migrations and comprehensive tests.
- Do not introduce Redis, Celery, Kafka, Kubernetes, AWS, or authentication yet.

Definition of done:

- Database schema/migrations work.
- Sandbox lifecycle works.
- Multiple executions can belong to one sandbox.
- Execution timeout does not incorrectly expire the sandbox.
- Expired sandbox cannot execute.
- Deletion cleans up owned execution resources.
- Restart/reconciliation behavior is covered.
- pytest, Ruff, and alembic checks pass.

PHASE 4 — AI-AGENT INTEGRATION

PHASE 4 — AI-AGENT INTEGRATION

Goal:

Make the sandbox platform directly usable as a structured tool by AI agents while preserving the existing architecture and security model.

Before implementation:

- Inspect the exact Phase 3 code and tests.
- Do not redesign the Docker engine or database model unless required.
- Define the minimum tool/API surface needed for an agent.

Expected direction:

- Provide clear machine-readable operations for creating a sandbox, executing code, inspecting execution results, inspecting sandbox state, and destroying a sandbox where appropriate.
- Keep request/response schemas deterministic and agent-friendly.
- Return concise structured errors that an agent can reason about.
- Define clear handling for expired, destroyed, timed-out, and failed states.
- Consider authentication/authorization only to the extent required by the agreed project scope.
- If a tool-calling protocol such as MCP is selected, isolate that adapter from the core sandbox service so the core API remains usable independently.
- Add integration tests for agent-style calls.
- Preserve all Phase 1-3 security guarantees.
- Do not add distributed infrastructure merely for demonstration.

Definition of done:

- An AI agent can invoke the sandbox functionality through the agreed structured interface.
- State and errors are unambiguous.
- Existing API behavior remains intact unless intentionally versioned.
- Tests and Ruff pass.
- Security regression tests pass.

PHASE 5 — CLOUD DEPLOYMENT AND OPERATIONS

PHASE 5 — CLOUD DEPLOYMENT AND OPERATIONS

Goal:

Deploy the sandbox platform to cloud infrastructure, initially targeting AWS EC2, without weakening the isolation model.

Requirements:

- Document the target AWS architecture before implementation.
- Separate application, database, and Docker execution concerns appropriately.
- Use environment-based configuration and cloud secret management rather than committed credentials.
- Provide reproducible deployment instructions.
- Configure persistent PostgreSQL appropriately for the chosen deployment model.
- Configure networking so the API is reachable while sandbox execution containers remain network-isolated.
- Add health/readiness checks.
- Add structured logging and basic metrics useful for monitoring execution, timeouts, failures, active sandboxes, and cleanup.
- Document resource sizing and operational limits.
- Preserve all sandbox security invariants.
- Do not expose Docker socket or host credentials to untrusted code.
- Do not claim production-grade multi-node scheduling unless it is actually implemented and tested.

Definition of done:

- A documented cloud deployment can be reproduced.
- Application health/readiness works.
- Database connectivity works.
- Sandbox execution remains isolated.
- Logs/metrics are useful.
- Secrets are managed safely.
- Local tests remain passing.